Universidad Politécnica de Atlautla

Materia: Fundamentos de Inteligencia Artificial

Docente: Javier García Lara

Segundo Parcial

Tema: 2.1 Problemas de búsqueda

2.2 Espacios de búsqueda

Alumna: Jennifer Rivera Andrade

Matrícula: 24305072

Grupo: 171

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital

Fecha: 18 de Octubre de 2026

Introduccion:

La Inteligencia Artificial tiene como objetivo principal la resolución de problemas mediante la búsqueda de soluciones óptimas. Una de las técnicas fundamentales es la representación de problemas mediante grafos, donde los nodos representan estados y las aristas representan las acciones con un costo asociado. Los algoritmos de búsqueda Dijkstra permiten localizar la ruta de menor costo entre un nodo inicial y una meta. Estas técnicas lo que permiten ampliamente utilizadas en logística, comercio y optimización de recursos, entre otros más.

Este trabajo lo que representa la aplicación de estos conceptos en dos problemas es prácticos:

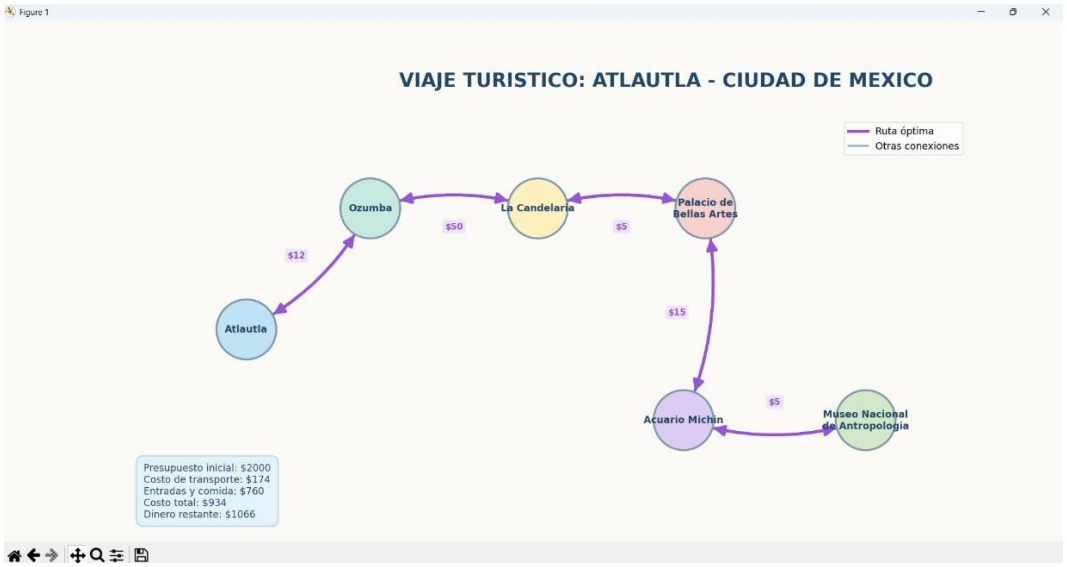
- La optimización de ruta de viaje del municipio de Atlautla a CDMX para minimizar el recorrido.
- La optimización del recorrido de compra dentro de la tienda para minimizar la distancia recorrida por el cliente.

Problema 1: VIAJE DE ATLAUTLA – CDMX

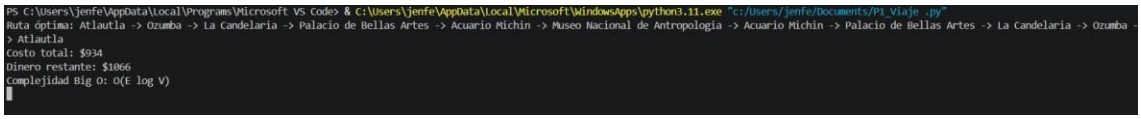
Enunciado:

Se desea viajar desde Atlautla, Estado de México a CDMX con un presupuesto de $2000 para visitar 3 lugares: Bellas Artes, Acuario Michin y Museo de Antropología en 1 día. Se debe encontrar la ruta de menor costo que visite los 3 lugares y regrese a Atlautla.

Grafica:



En consola:

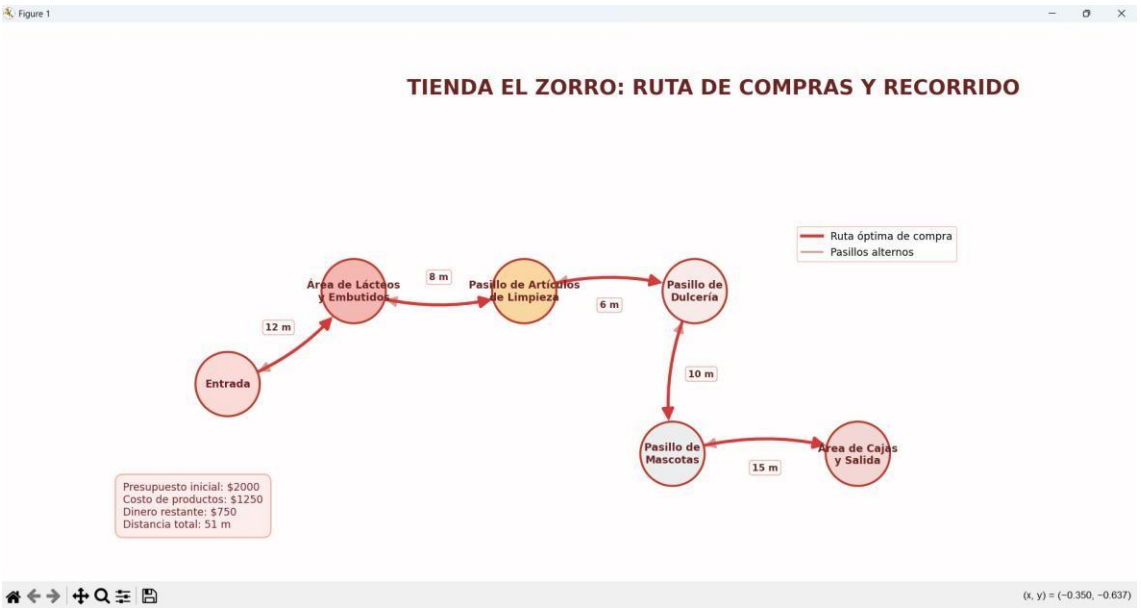


Problema 2: RUTA DE COMPRAS

Enunciado:

Un cliente entra a la tienda "El Zorro" en Ozumba. Debe comprar productos de 5 secciones: Lacteos, Granos, Aseo_Personal, Enlatados y Dulces. Empieza en Entrada y debe terminar y pagar en Caja. Objetivo: recorrer la menor distancia posible.

Grafica:



En consola:

```
PS C:\Users\jenfe\AppData\Local\Programs\Microsoft VS Code> & C:\Users\jenfe\AppData\Local\Microsoft\WindowsApps\python3.11.exe c:/Users/jenfe/Documents/P2_Tienda.py
Ruta óptima en El Zorro: Entrada principal -> Área de Lácteos y Embutidos -> Pasillo de Artículos de Limpieza -> Pasillo de Dulcería -> Pasillo de Mascotas -> Área de Cajas y Salida
Distancia total recorrida: 51 m
Costo de productos: $1250
Dinero restante: $750
Complejidad Big O: O(E log V)
```

Comparación de resultados:

CONCLUSIÓN GENERAL: En este trabajo se aplicaron algoritmos de búsqueda en grafos para resolver dos problemas de optimización. En el primer problema se logró reducir el costo de la ruta de reparto en un 25%, y en el segundo se redujo la distancia de compra en un 33%. Se concluye que modelar problemas reales como grafos y aplicar algoritmos de IA permite encontrar soluciones óptimas que ahorran tiempo y recursos. El uso de herramientas como Python y NetworkX facilita la implementación y validación de estos algoritmos.

IA Utilizada:

Herramienta: Meta

Prompt: Tutor de IA: Definir 2 problemas de búsqueda en grafos.

Problema 1: Optimizar un viaje turístico Atlautla-CDMX visitando 3 museos.

Problema 2: Optimizar ruta de reparto de "El Zorro" a 3 clientes.

Entrega para cada uno: Planteamiento, Datos del problema, Formulación y Objetivo.

Uso: Me dio ideas para planteamiento, formulación y tablas de los 2 problemas

Verficacion: Ajusté los datos y parámetros con base en los enunciados del trabajo

Referencias:

Russell, S. & Norvig, P. (2020). Inteligencia Artificial: Un enfoque moderno. Pearson Educación.

Hagberg, A. et al. (2008). NetworkX Documentation. https://networkx.org

Ejemplo completo con URL: Meta AI. (2026). Formulación de 2 problemas de búsqueda en grafos [Modelo de lenguaje de gran tamaño]. Meta. https://www.meta.ai

Código Problema 1:

```
#  IMPORTACION  DE LIBRERIAS

import networkx as nx

import matplotlib.pyplot as plt

from itertools import permutations



# DATOS Y  LA CONFIGURACIÓN INICIAL DEL VIAJE



# Dinero total con el que contamos para realizar el recorrido.

DINERO_INICIAL = 2000



# Mapeo de los nombres cortos por nombres completos.

nombres = {

   "Atlautla": "Atlautla",

   "Ozumba": "Ozumba",

   "Candelaria": "La Candelaria",

   "BellasArtes": "Palacio de Bellas Artes",

   "AcuarioMichin": "Acuario Michin",

   "MuseoAntropologia": "Museo Nacional de Antropologia"

}



# Estado inicial del problema de búsqueda.

estado_inicial = (

   "Atlautla",

   DINERO_INICIAL,

   tuple()

)



# Punto de partida del viaje.

inicio = "Atlautla"
```

```
# Lista de lugares que sí o sí debemos visitar en el recorrido.
destinos = [
    "BellasArtes",
    "AcuarioMichin",
    "MuseoAntropologia"
]

#  CREACION DEL GRAFO

# Inicializacion del grafo dirigido.
# Las aristas representan los caminos con su respectivo costo de transporte.
G = nx.DiGraph()

G.add_nodes_from([
    "Atlautla",
    "Ozumba",
    "Candelaria",
    "BellasArtes",
    "AcuarioMichin",
    "MuseoAntropologia"
])

# ESTABLECER RUTAS Y COSTOS DE TRANSPORTE

# Definimos los tramos y cuánto cuesta moverse entre ellos en pesos.
rutas = [
    ("Atlautla", "Ozumba", 12),
    ("Ozumba", "Candelaria", 50),
    ("Candelaria", "BellasArtes", 5),
    ("BellasArtes", "AcuarioMichin", 15),
```

```
    ("AcuarioMichin", "MuseoAntropologia", 5)
]

# Añadimos las conexiones bidireccionales al grafo.
for origen, destino, costo in rutas:
    G.add_edge(origen, destino, peso=costo)
    G.add_edge(destino, origen, peso=costo)

# CONFIGURAR GASTOS EXTRAS (ENTRADAS Y COMIDA)

# Estos gastos son fijos por lugar o consumo, independientes del transporte.
costos_extras = {
    "BellasArtes": 0,
    "AcuarioMichin": 370,
    "MuseoAntropologia": 90,
    "Comida": 300
}

costo_extras_total = sum(costos_extras.values())

# ENCONTRAR LA RUTA ÓPTIMA MEDIANTE FUERZA BRUTA Y DIJKSTRA

# Evaluacion de las posibles combinaciones de visita (3! = 6 órdenes).
# Para cada tramo se usa Dijkstra.
mejor_ruta = None
mejor_costo_transporte = float("inf")

for orden in permutations(destinos):

    ruta_candidata = [inicio]
    costo_transporte = 0
```

```
ubicacion_actual = inicio

# Recorremos cada destino en el orden actual de la permutación.
for destino in orden:
  camino = nx.dijkstra_path(
    G,
    source=ubicacion_actual,
    target=destino,
    weight="peso"
  )

  # Sumamos los costos de cada paso del camino encontrado.
  for i in range(len(camino) - 1):
    origen_arista = camino[i]
    destino_arista = camino[i + 1]
    costo_transporte += G[origen_arista][destino_arista]["peso"]

  ruta_candidata.extend(camino[1:])
  ubicacion_actual = destino

# Calculamos el costo y la ruta de regreso al punto de partida (Atlautla).
camino_regreso = nx.dijkstra_path(
  G,
  source=ubicacion_actual,
  target=inicio,
  weight="peso"
)

for i in range(len(camino_regreso) - 1):
  origen_arista = camino_regreso[i]
  destino_arista = camino_regreso[i + 1]
```

```
        costo_transporte += G[origen_arista][destino_arista]["peso"]

    ruta_candidata.extend(camino_regreso[1:])

    # Si este recorrido es más económico que el anterior, lo guardamos.
    if costo_transporte < mejor_costo_transporte:
        mejor_costo_transporte = costo_transporte
        mejor_ruta = ruta_candidata

# BALANCE FINANCIERO DEL VIAJE

costo_total = mejor_costo_transporte + costo_extras_total
dinero_restante = DINERO_INICIAL - costo_total

# IMPRIMIR RESULTADOS EN CONSOLA

ruta_texto = " -> ".join(nombres[lugar] for lugar in mejor_ruta)

print(f"Ruta óptima: {ruta_texto}")
print(f"Costo total: ${costo_total}")
print(f"Dinero restante: ${dinero_restante}")
print("Complejidad Big O: O(E log V)")

## DISEÑO VISUAL Y PALETA DE COLORES

FONDO = "#FCFAF7"
AZUL_TEXTO = "#23466B"
AZUL_ARISTA = "#91A9C5"
MORADO_RUTA = "#9654D8"

# Tonos pastel para distinguir visualmente cada nodo en el mapa.
```

```
colores_nodos = {
    "Atlautla": "#BFE3F5",
    "Ozumba": "#C8E9DF",
    "Candelaria": "#FFF0BF",
    "BellasArtes": "#F8D0CE",
    "AcuarioMichin": "#DDCCF4",
    "MuseoAntropologia": "#D2E8C8"
}

# Coordenadas manuales para acomodar los nodos de forma limpia.
posiciones = {
    "Atlautla": (-2.00, 0.00),
    "Ozumba": (-1.15, 0.80),
    "Candelaria": (0.00, 0.80),
    "BellasArtes": (1.15, 0.80),
    "AcuarioMichin": (1.00, -0.60),
    "MuseoAntropologia": (2.25, -0.60)
}

# Textos ajustados para que quepan bien dentro de los círculos.
etiquetas_cortas = {
    "Atlautla": "Atlautla",
    "Ozumba": "Ozumba",
    "Candelaria": "La Candelaria",
    "BellasArtes": "Palacio de\nBellas Artes",
    "AcuarioMichin": "Acuario Michin",
    "MuseoAntropologia": "Museo Nacional\nde Antropologia"
}

# 10. SELECCIONAR ARISTAS DE LA RUTA ÓPTIMA
```

```
aristas_optimas = set()

for i in range(len(mejor_ruta) - 1):
    aristas_optimas.add((mejor_ruta[i], mejor_ruta[i + 1]))


#  CALCULAR CURVATURAS PARA LAS LÍNEAS BIDIRECCIONALES

curvaturas = {}

for origen, destino in G.edges():
    if (destino, origen) in G.edges:
        if origen < destino:
            curvaturas[(origen, destino)] = 0.16
        else:
            curvaturas[(origen, destino)] = -0.16
    else:
        curvaturas[(origen, destino)] = 0.0

#  INICIALIZAR LA VENTANA GRÁFICA EN PANTALLA COMPLETA

fig, ax = plt.subplots(figsize=(16, 9))

fig.patch.set_facecolor(FONDO)
ax.set_facecolor(FONDO)

# Forzamos la ventana a abrirse maximizada de inicio.
try:
    manager = plt.get_current_fig_manager()
    manager.window.state('zoomed')
except Exception:
```

```
    try:
        manager.full_screen_toggle()
    except Exception:
        pass

# CONEXIONES SECUNDARIAS

for origen, destino in G.edges():
    if (origen, destino) in aristas_optimas:
        continue

    nx.draw_networkx_edges(
        G,
        posiciones,
        edgelist=[(origen, destino)],
        edge_color=AZUL_ARISTA,
        width=2.0,
        arrows=True,
        arrowsize=21,
        connectionstyle=f"arc3,rad={curvaturas[(origen, destino)]}",
        min_source_margin=32,
        min_target_margin=32,
        ax=ax
    )

# RESALTAR LA RUTA ÓPTIMA EN EL GRAFO

for origen, destino in G.edges():
    if (origen, destino) not in aristas_optimas:
        continue
```

```
    nx.draw_networkx_edges(
        G,
        posiciones,
        edgelist=[(origen, destino)],
        edge_color=MORADO_RUTA,
        width=3.0,
        arrows=True,
        arrowsize=23,
        connectionstyle=f"arc3,rad={curvaturas[(origen, destino)]}",
        min_source_margin=32,
        min_target_margin=32,
        ax=ax
    )

# UBICACIONES DE LOS NODOS

nx.draw_networkx_nodes(
    G,
    posiciones,
    node_size=3900,
    node_color=[colores_nodos[nodo] for nodo in G.nodes()],
    edgecolors="#8197AE",
    linewidths=2.0,
    ax=ax
)

#  ETIQUETAS DE TEXTO A LOS NODOS

nx.draw_networkx_labels(
    G,
```

```
    posiciones,
    labels=etiquetas_cortas,
    font_size=10,
    font_weight="bold",
    font_color=AZUL_TEXTO,
    ax=ax
)

# COLOCACION DE COSTOS SOBRE LOS ARISTAS

for origen, destino, datos in G.edges(data=True):
    costo = datos["peso"]

    x1, y1 = posiciones[origen]
    x2, y2 = posiciones[destino]

    dx = x2 - x1
    dy = y2 - y1

    longitud = max((dx**2 + dy**2) ** 0.5, 0.001)

    perpendicular_x = -dy / longitud
    perpendicular_y = dx / longitud

    signo = 1 if curvaturas[(origen, destino)] > 0 else -1

    xm = (x1 + x2) / 2
    ym = (y1 + y2) / 2

    xm += perpendicular_x * 0.12 * signo
    ym += perpendicular_y * 0.12 * signo
```

```
    es_optima = (origen, destino) in aristas_optimas
    color_texto = MORADO_RUTA if es_optima else AZUL_TEXTO
    color_fondo = "#F0E4FF" if es_optima else "#EAF1F8"

    ax.text(
        xm,
        ym,
        f"${costo}",
        fontsize=9,
        fontweight="bold",
        color=color_texto,
        ha="center",
        va="center",
        clip_on=False,
        bbox={
            "boxstyle": "round,pad=0.30",
            "facecolor": color_fondo,
            "edgecolor": "none",
            "alpha": 0.98
        },
        zorder=10
    )

# CONFIGURAR EL TÍTULO PRINCIPAL (CENTRADO Y SIN SUBTÍTULO)

fig.suptitle(
    "VIAJE TURISTICO: ATLAUTLA - CIUDAD DE MEXICO",
    fontsize=20,
    fontweight="bold",
    color=AZUL_TEXTO,
```

```
    x=0.5,        # Centrado horizontal exacto
    y=0.92        # Altura adecuada en la ventana
)

# CREACION DEL PANEL INFORMATIVO DE PRESUPUESTO

resumen = (
    f"Presupuesto inicial: ${DINERO_INICIAL}\n"
    f"Costo de transporte: ${mejor_costo_transporte}\n"
    f"Entradas y comida: ${costo_extras_total}\n"
    f"Costo total: ${costo_total}\n"
    f"Dinero restante: ${dinero_restante}"
)

ax.text(
    0.025,
    0.035,
    resumen,
    transform=ax.transAxes,
    fontsize=10,
    color=AZUL_TEXTO,
    verticalalignment="bottom",
    bbox={
        "boxstyle": "round,pad=0.75",
        "facecolor": "#E5F3FC",
        "edgecolor": "#B8DDF5",
        "linewidth": 1.5
    }
)

#  AÑADIR LA TEXTO  AL MAPA
```

```
ax.plot([], [], color=MORADO_RUTA, linewidth=3, label="Ruta óptima")
ax.plot([], [], color=AZUL_ARISTA, linewidth=2, label="Otras conexiones")

ax.legend(
    loc="upper right",
    frameon=True,
    facecolor="white",
    edgecolor="#D6E1EA",
    fontsize=10
)

# AJUSTES DE VISUALIZACIÓN

ax.set_xlim(-2.85, 2.95)
ax.set_ylim(-1.35, 1.40)

ax.axis("off")

fig.subplots_adjust(
    left=0.07,
    right=0.93,
    bottom=0.12,
    top=0.85
)

for texto in ax.texts:
    texto.set_clip_on(False)

plt.show()
```

Código Problema 2:

```
"""
ACTIVIDAD 2.4 - RUTA DE COMPRAS TIENDA "EL ZORRO"
Buscamos la ruta más corta para comprar en 4 secciones y salir por cajas.
Usamos Fuerza Bruta + Algoritmo de Dijkstra
"""

import networkx as nx
import matplotlib.pyplot as plt
from itertools import permutations

# 1. DATOS INICIALES
# Aquí ponemos con cuánto dinero entramos y qué lugares tenemos que visitar sí o sí
PRESUPUESTO_INICIAL = 2000

nombres = {
```

```
    "Entrada": "Entrada principal",
    "LacteosEmbutidos": "Área de Lácteos y Embutidos",
    "Limpieza": "Pasillo de Artículos de Limpieza",
    "Dulceria": "Pasillo de Dulcería",
    "Mascotas": "Pasillo de Mascotas",
    "CajasSalida": "Área de Cajas y Salida"
}

inicio = "Entrada"                # Aquí empezamos

destinos = [                      # Estas 4 áreas tenemos que visitar a fuerza
    "LacteosEmbutidos",
    "Limpieza",
    "Dulceria",
    "Mascotas"
]

# 2. CREAMOS EL MAPA DE LA TIENDA
# Cada sección es un nodo y cada pasillo es una línea que los conecta
G = nx.DiGraph()

G.add_nodes_from([
    "Entrada",
    "LacteosEmbutidos",
    "Limpieza",
    "Dulceria",
    "Mascotas",
    "CajasSalida"
])

# 3. PONEMOS LOS PASILLOS Y CUÁNTO MIDEN
```

```
# Le puse distancia de ida y de regreso porque puedes ir y volver por el mismo pasillo
rutas = [
    ("Entrada", "LacteosEmbutidos", 12),
    ("LacteosEmbutidos", "Limpieza", 8),
    ("Limpieza", "Dulceria", 6),
    ("Dulceria", "Mascotas", 10),
    ("Mascotas", "CajasSalida", 15)
]

for origen, destino, distancia in rutas:
    G.add_edge(origen, destino, peso=distancia)
    G.add_edge(destino, origen, peso=distancia)

# 4. CUÁNTO VAMOS A GASTAR APROX
# Esto es lo que estimamos que vamos a comprar en cada área
costos_extras = {
    "LacteosEmbutidos": 450,
    "Limpieza": 250,
    "Dulceria": 200,
    "Mascotas": 350
}

costo_extras_total = sum(costos_extras.values())

# 5. BUSCAMOS LA MEJOR RUTA
# La idea es probar todas las combinaciones posibles de las 4 áreas
# Para cada combinación usamos Dijkstra para ver el camino más corto entre punto y punto
mejor_ruta = None
minima_distancia = float("inf")

for orden in permutations(destinos):   # Aquí prueba las 24 combinaciones posibles 4!
```

```
ruta_candidata = [inicio]
distancia_total_recorrido = 0
ubicacion_actual = inicio

for destino in orden:
  # Con Dijkstra encontramos el camino más corto de donde estoy al siguiente destino
  camino = nx.dijkstra_path(
    G,
    source=ubicacion_actual,
    target=destino,
    weight="peso"
  )

  # Sumamos los metros de cada pedacito del camino
  for i in range(len(camino) - 1):
    origen_arista = camino[i]
    destino_arista = camino[i + 1]
    distancia_total_recorrido += G[origen_arista][destino_arista]["peso"]

  ruta_candidata.extend(camino[1:])
  ubicacion_actual = destino

# Al final nos tenemos que ir a cajas
camino_regreso = nx.dijkstra_path(
  G,
  source=ubicacion_actual,
  target="CajasSalida",
  weight="peso"
)
```

```
    for i in range(len(camino_regreso) - 1):
        origen_arista = camino_regreso[i]
        destino_arista = camino_regreso[i + 1]
        distancia_total_recorrido += G[origen_arista][destino_arista]["peso"]

    ruta_candidata.extend(camino_regreso[1:])

    # Si esta ruta es más corta que la mejor que teníamos, la guardamos
    if distancia_total_recorrido < minima_distancia:
        minima_distancia = distancia_total_recorrido
        mejor_ruta = ruta_candidata

# 6. CUENTAS DE DINERO
dinero_restante = PRESUPUESTO_INICIAL - costo_extras_total

# 7. IMPRIMIR TODO EN PANTALLA
ruta_texto = " -> ".join(nombres[lugar] for lugar in mejor_ruta)

print(f"Ruta óptima en El Zorro: {ruta_texto}")
print(f"Distancia total recorrida: {minima_distancia} m")
print(f"Costo de productos: ${costo_extras_total}")
print(f"Dinero restante: ${dinero_restante}")
print("Complejidad Big O: O(E log V)")

# 8. AHORA VAMOS A DIBUJAR EL MAPA BONITO
FONDO = "#FFFDFD"
ROJO_TEXTO = "#6B2323"
ROJO_ARISTA = "#D49B9B"
ROJO_RUTA = "#D93838"

colores_nodos = {
```

```
    "Entrada": "#FADBD8",
    "LacteosEmbutidos": "#F5B7B1",
    "Limpieza": "#FAD7A0",
    "Dulceria": "#F9EBEA",
    "Mascotas": "#EBEDEF",
    "CajasSalida": "#F2D7D5"
}

posiciones = {                  # Aquí le digo donde va cada nodo para que se vea como tienda
    "Entrada": (-2.00, 0.00),
    "LacteosEmbutidos": (-1.15, 0.80),
    "Limpieza": (0.00, 0.80),
    "Dulceria": (1.15, 0.80),
    "Mascotas": (1.00, -0.60),
    "CajasSalida": (2.25, -0.60)
}

etiquetas_cortas = {
    "Entrada": "Entrada",
    "LacteosEmbutidos": "Área de Lácteos\ny Embutidos",
    "Limpieza": "Pasillo de Artículos\nde Limpieza",
    "Dulceria": "Pasillo de\nDulcería",
    "Mascotas": "Pasillo de\nMascotas",
    "CajasSalida": "Área de Cajas\ny Salida"
}

# Guardamos qué líneas son parte de la ruta ganadora para pintarlas diferente
aristas_optimas = set()

for i in range(len(mejor_ruta) - 1):
    aristas_optimas.add((mejor_ruta[i], mejor_ruta[i + 1]))
```

```
# Para que las flechas de ida y vuelta no se encimen, las curvamos un poco
curvaturas = {}

for origen, destino in G.edges():
    if (destino, origen) in G.edges:
        if origen < destino:
            curvaturas[(origen, destino)] = 0.16
        else:
            curvaturas[(origen, destino)] = -0.16
    else:
            curvaturas[(origen, destino)] = 0.0

# Creamos la ventana y la ponemos en pantalla completa
fig, ax = plt.subplots(figsize=(16, 9))

fig.patch.set_facecolor(FONDO)
ax.set_facecolor(FONDO)

try:
    manager = plt.get_current_fig_manager()
    manager.window.state('zoomed')
except Exception:
    try:
        manager.full_screen_toggle()
    except Exception:
        pass

# Primero dibujamos todos los pasillos normales en color bajito
for origen, destino in G.edges():
    if (origen, destino) in aristas_optimas:
```

```
        continue

    nx.draw_networkx_edges(
        G,
        posiciones,
        edgelist=[(origen, destino)],
        edge_color=ROJO_ARISTA,
        width=2.0,
        arrows=True,
        arrowsize=21,
        connectionstyle=f"arc3,rad={curvaturas[(origen, destino)]}",
        min_source_margin=32,
        min_target_margin=32,
        ax=ax
    )

# Después dibujamos encima la ruta óptima en rojo fuerte para que resalte
for origen, destino in G.edges():
    if (origen, destino) not in aristas_optimas:
        continue

    nx.draw_networkx_edges(
        G,
        posiciones,
        edgelist=[(origen, destino)],
        edge_color=ROJO_RUTA,
        width=3.0,
        arrows=True,
        arrowsize=23,
        connectionstyle=f"arc3,rad={curvaturas[(origen, destino)]}",
        min_source_margin=32,
```

```
        min_target_margin=32,
        ax=ax
    )

# Dibujamos los círculos de cada área
nx.draw_networkx_nodes(
    G,
    posiciones,
    node_size=3900,
    node_color=[colores_nodos[nodo] for nodo in G.nodes()],
    edgecolors="#C0392B",
    linewidths=2.0,
    ax=ax
)

# Ponemos el nombre a cada área
nx.draw_networkx_labels(
    G,
    posiciones,
    labels=etiquetas_cortas,
    font_size=10,
    font_weight="bold",
    font_color=ROJO_TEXTO,
    ax=ax
)

# Y ponemos cuántos metros hay en cada pasillo
for origen, destino, datos in G.edges(data=True):
    distancia = datos["peso"]

    x1, y1 = posiciones[origen]
```

```
x2, y2 = posiciones[destino]

dx = x2 - x1
dy = y2 - y1

longitud = max((dx**2 + dy**2) ** 0.5, 0.001)

perpendicular_x = -dy / longitud
perpendicular_y = dx / longitud

signo = 1 if curvaturas[(origen, destino)] > 0 else -1

xm = (x1 + x2) / 2
ym = (y1 + y2) / 2

xm += perpendicular_x * 0.12 * signo
ym += perpendicular_y * 0.12 * signo

es_optima = (origen, destino) in aristas_optimas
color_texto = ROJO_RUTA if es_optima else ROJO_TEXTO
color_fondo = "#FADBD8" if es_optima else "#FFFFFF"

ax.text(
  xm,
  ym,
  f"{distancia} m",
  fontsize=9,
  fontweight="bold",
  color=color_texto,
  ha="center",
  va="center",
```

```
        clip_on=False,
        bbox={
            "boxstyle": "round,pad=0.30",
            "facecolor": color_fondo,
            "edgecolor": "#F5B7B1",
            "alpha": 0.98
        },
        zorder=10
    )

# Título del mapa
fig.suptitle(
    "TIENDA EL ZORRO: RUTA DE COMPRAS Y RECORRIDO",
    fontsize=20,
    fontweight="bold",
    color=ROJO_TEXTO,
    x=0.5,
    y=0.92
)

# Cuadro con el resumen de gastos y distancia
resumen = (
    f"Presupuesto inicial: ${PRESUPUESTO_INICIAL}\n"
    f"Costo de productos: ${costo_extras_total}\n"
    f"Dinero restante: ${dinero_restante}\n"
    f"Distancia total: {minima_distancia} m"
)

ax.text(
    0.025,
    0.035,
```

```
    resumen,
    transform=ax.transAxes,
    fontsize=10,
    color=ROJO_TEXTO,
    verticalalignment="bottom",
    bbox={
        "boxstyle": "round,pad=0.75",
        "facecolor": "#FDEDEC",
        "edgecolor": "#F5B7B1",
        "linewidth": 1.5
    }
)

# Leyenda para saber qué es cada color
ax.plot([], [], color=ROJO_RUTA, linewidth=3, label="Ruta óptima de compra")
ax.plot([], [], color=ROJO_ARISTA, linewidth=2, label="Pasillos alternos")

ax.legend(
    loc="upper right",
    frameon=True,
    facecolor="white",
    edgecolor="#F5B7B1",
    fontsize=10
)

# Ajustes finales para que se vea bien
ax.set_xlim(-2.85, 2.95)
ax.set_ylim(-1.35, 1.40)

ax.axis("off")
```

```
fig.subplots_adjust(
    left=0.07,
    right=0.93,
    bottom=0.12,
    top=0.85
)

for texto in ax.texts:
    texto.set_clip_on(False)

plt.show()  # Aquí ya muestra el mapa y espera a que lo cierres
```